Ders 13 Genişletilmiş Notu: Video Kiralama Örneğinde Adım Adım Refactoring ve Karşıt Kalıplar

Ders 13 Genişletilmiş Notu: Video Kiralama Örneğinde Adım Adım Refactoring ve Karşıt Kalıplar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yeri ve Yöntemi
- 2. Başlangıç Kodu: Video Kiralama Uygulaması
- 3. Refactoring'in Sınırı: Geriye Doğru Uyumluluk
- 4. Eclipse'in Otomatik Refactoring Araçları Üzerine
- 5. 1. Adım: switch Bloğunu amountFor Metoduna Çıkarmak
- 6. Testler: Aynı Test Kodunun Her Adımda Geçmesi
- 7. 2. Adım: Değişken Adlarını Düzenlemek
- 8. 3. Adım: amountFor Metodunu Rental'a Taşımak (getCharge)
- 9. Kara Kutu ve Beyaz Kutu Sınama
- 10. 4. Adım: thisAmount Geçici Değişkenini Kaldırmak
- 11. 5. Adım: Bonus Hesabını Rental'a Taşımak
- 12. 6. Adım: getTotalCharge ve getTotalBonus
- 13. 7. ve 8. Adımlar: Hesapların Movie Sınıfına Taşınması
- 14. 9. Adım: Etkileşim Şeması ve getMovieTitle
- 15. 10. Adım: Çok Biçimlilik ya da Price Sınıfları
- 16. Karşıt Kalıplar (Anti-Patterns)
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Yeri ve Yöntemi

Önceki mevcut kayıtta (ders 11) kod kusurları (code smells) ve onları düzelten refactoring eylemleri tamamlanmıştı; son konu Feature Envy idi ve slaytın kalan kısmı bütüncül bir refactoring örneğine ayrılmıştı. Bu derste o bütüncül örnek yürütüldü: şimdiye kadar tek tek görülen kod kusurlarını tespit etme ve onlara karşılık gelen refactoring eylemlerini uygulama işi, tek bir örnek üzerinde baştan sona yapıldı.

Örnek, kod kusurları anlatılırken de sürekli kullanılan video kaset kiralama uygulamasıdır. Aynı uygulama en kötü halinden başlanıp adım adım iyileştirilir. Hoca, kendi aklına gelen ilk sırayla ilerlerse ders notlarındaki sıradan sapabileceğini ve bazı adımları anlatmayı unutabileceğini, bu yüzden Fowler'ın kitabındaki sırayı izleyeceğini söyledi. En sondaki tasarım tartışmasında ise kendi görüşlerini ayrıca ekleyeceğini belirtti.

Kodlar ders paylaşım alanındaki Eclipse projesinde, `refactoring` dizininin altında paketler halinde durur. `fowler_00` ilk haldir. Her refactoring eylemi bir sonraki paketi verir: `fowler_00` üzerinde bir değişiklik yapılınca `fowler_01`, onun üzerinde bir değişiklik yapılınca `fowler_02` elde edilir ve bu böyle `fowler_09`'a kadar gider; ayrıca `fowler_10_alt` ve `fowler_11_alt` adlı alternatif paketler vardır (ekrandan, 00:10:00). Her paketin testleri `testing` dizininde aynı adlı pakette durur (`TestAll`, `TestCustomer`, `TestMovie`). Hoca iki ardışık paketi Eclipse'in karşılaştırma (compare) aracıyla yan yana açarak her adımda neyin değiştiğini gösterdi.

Refactoring'in yöntemi hatırlatıldı: küçük değişiklikler yapılır, ama küçük bir değişiklik bile kodda bir yeri bozabilir; bu yüzden her refactoring eyleminden sonra testler çalıştırılır. Testler `Customer` ve `Movie` sınıflarının bütün metotlarını sınar ve daha önce JUnit örneğinde ayrıntılı incelenmişti.

2. Başlangıç Kodu: Video Kiralama Uygulaması

Sınıf yapısı şöyledir: `Customer` sınıfı kiralama yapan müşteriyi, `Movie` sınıfı kiralanan filmi, `Rental` sınıfı bir kira kaydını modeller. Bir müşteri, liste halinde birçok `Rental` nesnesiyle ilişkilidir; bir `Rental` nesnesi de bir `Movie` ile ilişkilidir. Slayttaki sınıf şeması şöyle özetlenebilir (ekrandan, 00:30:00):

```
Movie    (- _priceCode : int)
  ^ 1
  | 1
Rental   (- _daysRented : int)
  ^ *
  | 1
Customer (+ statement(), - amountFor(Rental))   <- 1. adımdan sonraki hal
```

`Customer` sınıfında müşterinin adı, kira kayıtlarının listesi, kurucu, kira kaydı ekleme metodu, ad için get metodu ve `statement` metodu vardır (ekrandan, 00:09:55):

```
public class Customer {
    private String _name;
    private Vector<Rental> _rentals = new Vector<Rental>();
    public Customer (String name){
        _name = name;
    }
    public void addRental(Rental arg) {
        _rentals.addElement(arg);
    }
    public String getName (){
        return _name;
    }
    public String statement( ) {
        double totalAmount = 0;
        int frequentRenterPoints = 0;
        Enumeration<Rental> rentals = _rentals.elements();
        String result = "Rental Record for " + getName() + "\n";
        while (rentals.hasMoreElements()) {
            double thisAmount = 0;
            Rental each = rentals.nextElement();
            //determine amounts for each line
            switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2.0;
                if (each.getDaysRented() > 2)
                    thisAmount += (each.getDaysRented() - 2) * 1.5;
                break;
            case Movie.NEW_RELEASE:
                thisAmount += each.getDaysRented() * 3.0;
                break;
            case Movie.CHILDRENS:
                thisAmount += 1.5;
                if (each.getDaysRented() > 3)
                    thisAmount += (each.getDaysRented() - 3) * 1.5;
                break;
            }
            // add frequent renter points
            frequentRenterPoints ++;
```

```
            // add bonus for a two day new release rental
            if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE)
                    && each.getDaysRented() > 1) frequentRenterPoints ++;
            //show figures for this rental
            result += "\t" + each.getMovie().getTitle()+ "\t" +
                String.valueOf(thisAmount) + "\n";
            totalAmount += thisAmount;
        }
        //add footer lines
        result += "Amount owed is " + totalAmount+"\n";
        result += "You earned " + frequentRenterPoints
                + " frequent renter points";
        return result;
    }
}
```

(Kod, 00:09:55, 00:10:00 ve 00:20:00 karelerinden birleştirildi.)

`statement` en kalabalık metottur; hoca onu "hatalar yumağı, kusurlar yumağı" diye niteledi ve 40 satıra yakın olduğunu söyledi. Bu yumak adım adım çözülecektir. İlk göze çarpan kusur, `while` içindeki `switch-case`'tir. Hoca `switch-case`'in nesneye dayalı tasarımda "en çok batan husus" olduğunu, burada kalıtım ve çok biçimlilikle daha iyi bir tasarıma gidilebileceğini söyledi. Bunun dışında bol bol Extract Method yapılacak, yinelenen kodlar bulunacak ve bazı işlerin yerinin `Customer` sınıfı olmadığı anlaşılacaktır.

Hoca burada sorumluluk atama ilkesini hatırlattı: bir işle ilgili sorumluluk, o işle ilgili veriye sahip olan sınıfa verilmelidir; sorumluluğu yerine getirmek için gereken verinin en çoğuna kim sahipse sorumluluk ona verilir.

`Movie` sınıfının başında public sabit üye alanlar vardır: `CHILDRENS = 2`, `REGULAR = 0`, `NEW_RELEASE = 1`; ayrıca `_title` ve `_priceCode` alanları, `Movie(String title, int priceCode)` kurucusu, `getPriceCode`, `setPriceCode` ve `getTitle` metotları (ekrandan, 01:20:00).

3. Refactoring'in Sınırı: Geriye Doğru Uyumluluk

Refactoring'in özü, dışarıya verilen hizmetleri, yani dışarıya açılan hizmet penceresini değiştirmeden iç yapıyı düzenlemektir. Hoca bu örneği sıfırdan tasarlasaydı film türleri `Movie`'nin alt sınıfları olurdu. Bu durumda her alt sınıf kendi film türünü temsil ettiği için kurucuya fiyat kodu vermenin hiçbir anlamı kalmazdı; tür zaten fiyat koduyla ilişkilidir. Ama `Movie`'nin kurucusuna fiyat kodu verilmesi ve public sabit üye alanlar, geriye doğru uyumluluğun önündeki en büyük engeldir. Alt sınıflara geçmek dışarıya verilen hizmetin şeklini değiştirebilir.

Bu noktaya gelindiğinde iki seçenek vardır: refactoring ilerletilir ama geriye doğru uyumluluk kaybedilir, ya da "buraya kadar getirdik" denip orada durulur. Geriye doğru uyumluluk önemliyse durulur, değilse devam edilir. Hoca dersin sonunda bu tartışmaya geri döndü (bölüm 15).

4. Eclipse'in Otomatik Refactoring Araçları Üzerine

Hoca Eclipse'in Refactor menüsünü açtı: Move (Move Method ya da Move Field, seçilen öğeye göre), Extract Method, Extract Superclass, kalıtımla ilgili yukarı ve aşağı taşıma eylemleri, Extract Class, uzun parametre listesi için Introduce Parameter Object gibi seçenekler vardır.

Ancak bunlara tek başına güvenilmemesi gerektiğini söyledi ve iki gerekçe verdi. Birincisi, okuduğu eski bir makaleye göre Eclipse'in yapabildiği refactoring eylemleri yaklaşık %70-80 oranında hatasızdır. Aradan yıllar geçtiği için bu oran artmış olabilir, ama yine de işi kendimiz yapmak daha doğrudur. Otomatik araç kullanılırsa testler mutlaka yeniden çalıştırılmalı ve sorun varsa geri dönülmelidir. İkincisi, hoca ders notlarını hazırlarken bu kodda Eclipse'in Extract Method'unu denemiş ve araç başarısız olmuştur. Nedeni geçici değişkenlerdir: birden fazla yerde geçen geçici değişkenler yüzünden çıkarma yapılamamış, değişkenlerin tanımlanma sırası değiştirilince başarılı olmuştur. Hoca derste de denemek istedi ama proje ayarları uzun sürdüğü için hazır paketler üzerinden ilerlemeyi seçti.

5. 1. Adım: switch Bloğunu amountFor Metoduna Çıkarmak

Gözüme en çok batan şey `statement` metodunun devasa olmasıdır. Nesneye dayalı yaklaşım parçalara ayırmayı önerir. İlk iş, bu büyük metodu ikiye bölmektir; en dikkat çeken kısım `switch` bloğudur. `switch` içindeki bütün kod kesilip `amountFor` adlı yeni bir metoda taşınır. `switch` her filmin bedelini hesapladığı için metodun döndüreceği şey, `thisAmount`'a eklenecek kira bedelidir; bu yüzden metodun dönüş tipi `double`'dır. Metot, her bir kira kaydının bedelini hesaplamak içindir.

Hoca `thisAmount` değişkeninin `switch`'e girmeden önce sıfırlandığını, sonra hangi `case` çalışırsa orada değer aldığını ve bir daha değişmediğini gösterdi. Bu, ileride Replace Temp with Query ile kaldırılabileceği anlamına gelir. Ama "adım adım" ilerlendiği için bu adımda `thisAmount` kalır. Karşılaştırmada yeni hal şöyledir (ekrandan, 00:21:20):

```
while (rentals.hasMoreElements()) {
    double thisAmount = 0;
    Rental each = (Rental) rentals.nextElement();
    thisAmount = amountFor(each);
    // add frequent renter points
    frequentRenterPoints ++;
    ...
}
```

Yeni `amountFor` metodunun içinde `switch` hâlâ durur; `thisAmount` geçici değişkeni de metodun içine taşınmıştır, hesaplanır ve `return` ile döndürülür. Metot `private` tanımlandı, çünkü dışarıya hizmet vermesi gerekmez.

Hoca bu sırada, `REGULAR` durumunda değişkenin önce 0 yapılıp sonra 2 eklenmesi yerine doğrudan 2 atanabileceğini, `NEW_RELEASE` durumunda ise doğrudan çarpım sonucunun döndürülebileceğini, yani bu geçici değişkenin de bir yerde kaldırılabileceğini söyledi. Ama adım adım gidildiği için şimdilik buna dokunulmadı.

Slaytta bu adımdan sonraki etkileşim (sequence) şeması da verilir (ekrandan, 00:30:00). Kullanıcı `aCustomer`'a `statement()` gönderir; `aCustomer` her kira kaydı için (`*[for all rentals]`) kendine `amountFor(aRental)` çağrısı yapar; bu çağrının içinde `aRental`'a `getMovie()`, `aMovie`'ye `getPriceCode()`, `aRental`'a `getDaysRented()` gönderilir. Ardından bonus hesabı için yine `getPriceCode()` ve `getDaysRented()`, makbuz satırı için `getTitle()` çağrılır. Şemadaki notlar: "Tek kira için yapılan hesaplar her döngüde biriktirilir", "Tek kira için ücret hesabı tamamlanır", "Tek kira için bonus hesabı tamamlanır", "Tek kira için makbuz satırı basımı tamamlanır", "Biriken bilgiyle makbuz basımı tamamlanır". Hoca bu şemanın şimdilik gerekmediğini, ama ileride bir şeyi fark etmemizi sağlayacağını söyledi.

6. Testler: Aynı Test Kodunun Her Adımda Geçmesi

Arayüz değişmediği için birim testleri aynen kullanılmaya devam eder. `fowler_00`'ın test kodu ile `fowler_01`'in test kodu, paket adı dışında birbirinin aynısıdır. Hoca bunu göstermek için `fowler_00` paketindeki `TestCustomer` kodunu kopyalayıp diğer paketlere yapıştırdı. Bu sırada Türkçe karakter kodlamasıyla ilgili küçük bir sorun çıktı ve dosyalar önceki sürümlerinden geri alındı; sonunda testler çalıştırıldı ve hiçbir hata alınmadı.

Hocanın vurguladığı nokta şudur: aynı test kodu yapıştırıldığı halde ne hata (bug) ne de derleme hatası çıktı. Çünkü test kodu sınamayı dışarıya sunulan pencere, yani arayüz üzerinden yapar. Hoca burada "arayüz" sözcüğünü Java'nın `interface` anlamında değil, API anlamında kullandığını belirtti. Arayüz değişmediğinde test kodunu değiştirmeye gerek yoktur. Beyaz kutu (white box) sınama yapılıyorsa, yeni dallardan da geçecek şekilde yeni test durumları hazırlamak ayrı bir tercihtir.

7. 2. Adım: Değişken Adlarını Düzenlemek

İkinci adımda `amountFor` metodundaki değişken adları düzenlendi. `each` adı, eski halinde liste içindeki her kira kaydına sırayla erişirken anlamlıydı. Ama artık metot tek bir kiranın bedelini hesaplıyor; "her biri" adı burada anlamlı değildir. Bu yüzden parametrenin adı `aRental` (bir kira kaydı) yapıldı; metot içindeki `thisAmount` da `result` oldu.

Soru: Bu da bir refactoring sayılır mı?

Cevap: Evet. Sonuçta anlaşılabilirliği iyileştirmek için yapılan bir düzenlemedir. Bu tür adlandırma değişiklikleri Eclipse'te de Refactor menüsünün altında yer alır.

Eclipse üzerinden yapılan basit bir ad değişikliği, ilgili bütün kullanımları düzelttiği için büyük olasılıkla doğru olacaktır. Yine de hoca test kodunu kopyalayıp bütün testleri çalıştırarak "içimizi rahatlatmak" gerektiğini söyledi.

Hoca burada sıralamayla ilgili genel bir ilke verdi: bir refactoring eylemi yeni bir kod kusurunu ortaya çıkarmış olabilir. Hangi sırayla yapılacağı sorulursa, en göze batan yerden başlanır; yapılan düzenlemenin yeni bir sorun getirip getirmediğine bakılır ve memnun olunana kadar zincirleme düzeltilmeye devam edilir.

8. 3. Adım: amountFor Metodunu Rental'a Taşımak (getCharge)

`amountFor` metoduna bakıldığında şu görülür: kira kaydının filmi alınır, filmin fiyat koduna bakılır, kira kaydının kaç gün kiralandığına bakılır. Metotta müşterinin kendisiyle ilgili hiçbir veri yoktur. Bütün veriler ya filmle ya da kiranın kaç gün sürdüğüyle ilgilidir. Demek ki bu metot `Customer`'a ait değildir; olması gereken yer `Rental` sınıfıdır.

Soru: Metotta filmle ilgili bilgi de var; neden `Movie`'ye değil de `Rental`'a?

Cevap: `Rental`'ın `Movie` ile sahiplik ilişkisi olduğu için filmle ilgili bilgiye de oradan ulaşılır. İlk aşamada `Rental`'a taşımak yeterlidir.

Metot taşınırken adı da değiştirildi: `amountFor` yerine `getCharge`. Hoca bunu bir konuşma gibi anlattı: "Ey kira kaydı, ödemem gereken bedel ne? Onu bana getir." Okuma amaçlı bir erişim olduğu için `get` önekiyle `getCharge` en mantıklı addır. Artık `Rental` içinde olduğu için `aRental.getMovie()` yerine doğrudan kendi `getMovie()` ve `getDaysRented()` metotları çağrılır. `fowler_02` ile `fowler_03` karşılaştırıldığında `Customer` içindeki tek fark şudur (ekrandan, 00:35:10):

```
// fowler_02
thisAmount = amountFor(aRental);
// fowler_03
thisAmount = aRental.getCharge();
```

Hoca `getCharge`'ı ilk taşıdığında `public` yapmadığını, erişim belirteci yazmayıp paket erişiminde bıraktığını söyledi: normalde onu çağıran `Customer` nesnesidir ve aynı pakette olanların erişmesi yeterlidir. Java'da bir şey yazılmazsa paket erişimi olur. İstenirse `public` de yapılabilir; sonraki paketlerde `public` yapılmıştır.

Bu taşıma uyumu (cohesion) da yükseltti, özellikle `Customer` sınıfında. Hoca, bağlaşım ve uyumla ilgili statik ölçülerin hangi metodun hangi üyeye eriştiğine bakarak hesaplandığını hatırlattı; bu ölçüler de taşımayla iyileşir. Ama metrikleri bir yana bırakırsak asıl kazanç şudur: sınıfa ait olmaması gereken bir sorumluluk, ait olması gereken yere taşındı; yani anlamsal olarak da uyum yükseldi.

9. Kara Kutu ve Beyaz Kutu Sınama

`Rental` sınıfı yeni bir metot (`getCharge`) kazandığına göre, bu metot için ayrı bir test durumu (test case) oluşturulmalı mıdır?

Hoca bunun yaklaşıma bağlı olduğunu söyledi. Beyaz kutu yaklaşımıyla çalışılıyorsa, yani sınanacak birimin iç işleyişi bilinip sınamalar ona göre belirleniyorsa, yeni bir test durumu eklemek gerekir. Kara kutu (black box) yaklaşımında ise yapı hiç bilinmez; yalnız verilen girdilere karşılık beklenen çıktıların elde edilip edilmediğine bakılır. Daha az masraflı olan kara kutu yaklaşımında mevcut testlerle devam edilir. Bu derste de böyle yapıldı.

10. 4. Adım: thisAmount Geçici Değişkenini Kaldırmak

`Customer.statement` içindeki geçici değişkenlere tek tek bakıldı:

- `totalAmount` kaldırılamaz: kümülatif bir toplamdır, adı üstünde bir yerde biriktirilerek alınması gerekir.
- `frequentRenterPoints` de öyledir: yapılan kiralamaya göre ek puan biriktirilir.
- `thisAmount` ise tamamen gereksizdir: bir kez değer atandıktan sonra iki yerde kullanılır ve hiç değişmez.

Bu yüzden Replace Temp with Query uygulanır: geçici değişken silinir ve onu gördüğümüz her yere sorgu metodunun çağrısı (`aRental.getCharge()`) yazılır. Hoca bunun kayda değer bir performans kaybı yaratmayacağını söyledi: metot bir döngü içinde fazladan çağrılmıyor, yalnız iki kez çağrılıyor ve içinde çok fazla iş yapılmıyor. Slayt 53 bu adımı şöyle özetler (ekrandan, 00:40:00): "Dördüncü adım: statement metodundaki yerel thisAmount değişkeni gereksizdir. Bu değişkene getCharge metodunun döndürdüğü değer atanmakta ve değişkenin geçerlilik süresince bu değişkenin değeri değişmemektedir. Bu nedenle thisAmount yerine getCharge kullanılmıştır. Bedel hesaplama daha ileride gerçekleştiği için etkileşim şeması değişecektir."

Karşılaştırmada `thisAmount` satırının kalktığı, makbuz satırında ve toplamda doğrudan `aRental.getCharge()` çağrıldığı görülür. Testler yine sorunsuz geçti.

11. 5. Adım: Bonus Hesabını Rental'a Taşımak

Kira bedeliyle ilgili kısımlar düzeltildikten sonra sıra bonus hesabına geldi. Bonus da benzer biçimde filmin türüne ve kaç gün kiralandığına göre değişir. Bu yüzden kira bedeli için yapılan refactoring eylemleri bonus için de yapılır: ilgili kısım çıkarılır (Extract Method) ve `Rental`'a taşınır (Move Method).

Hoca "frequent renter points" adını da değiştirdi. "Bonus" sözcüğü video oyunlarındaki bonus seviyelerinden beri hem Türkçeye hem İngilizceye yerleşmiştir ve daha net, daha özdür. Bu yüzden ad `getBonus` oldu; ad değişikliği de bir refactoring'dir.

`statement` içinde artık "kira kaydı, bana kazanacağım bonusu ver" denir. `Rental`'daki yeni metot şöyledir (ekrandan, 00:44:45):

```
public int getBonus( ) {
    if ((getMovie().getPriceCode() == Movie.NEW_RELEASE)
            && getDaysRented() > 1)
        return 2;
    else
        return 1;
}
```

İş mantığı şudur: normal kiralamalardan 1 puan kazanılır; ama yeni çıkan bir film bir günden uzun kiralandıysa 1 puan daha, yani toplam 2 puan kazanılır. Eski kod önce sayacı bir artırıp sonra koşul doğruysa bir daha artırıyordu; böyle yazılsaydı `getBonus` içinde de bir geçici değişken gerekecekti. "Koşul gerçekleşiyorsa 2, yoksa 1 puan" biçimi daha derli toplu bir hesaplamadır ve geçici değişkeni ortadan kaldırır.

`statement` içindeki bonus toplamı yine kalır, çünkü o da kümülatiftir; ama artık daha az yerde kullanılır. Testler sorunsuz geçti. Sonuçta `Rental`'ın sorumluluğu arttı: hem bedeli hem bonusu hesaplar.

12. 6. Adım: getTotalCharge ve getTotalBonus

`statement`'taki toplam bedel ve toplam bonus değişkenleri kümülatif oldukları için tam olarak kaldırılamaz, ama oradan çıkarılabilir. Hoca amacın illa geçici değişkenleri kaldırmak olmadığını vurguladı. Asıl dert, `statement` metodunun diğer metotlara göre hâlâ en uzun metotlardan biri olmasıdır; bu yüzden onu nasıl kısaltacağına bakmaya devam etti. "Böl, parçala, yönet" ilkesiyle toplam bedelin ve toplam bonusun hesaplanması için `Customer` sınıfında yeni metotlar çıkarıldı (ekrandan, 00:48:50 ve 00:50:00):

```
public String statement( ) {
    Enumeration<Rental> rentals = _rentals.elements();
    String result = "Rental Record for " + getName() + "\n";
    while (rentals.hasMoreElements()) {
```

```
        Rental aRental = (Rental) rentals.nextElement();
        //show figures for this rental
        result += "\t" + aRental.getMovie().getTitle()+ "\t" +
            String.valueOf(aRental.getCharge()) + "\n";
    }
    //add footer lines
    result += "Amount owed is " + getTotalCharge() + "\n";
    result += "You earned " + getTotalBonus()
            + " frequent renter points";
    return result;
}
public double getTotalCharge( ) {
    double result = 0;
    for( Rental each : _rentals )
        result += each.getCharge();
    return result;
}
public int getTotalBonus( ) {
    int result = 0;
    for( Rental each : _rentals )
        result += each.getBonus( );
    return result;
}
```

`getTotalCharge` içinde `result` adlı bir geçici değişken hâlâ vardır. Ama hoca geçici değişkenlerin en önemli dezavantajının metodu uzatma eğilimi olduğunu söyledi. Bu metot kısadır ve kümülatif toplam olduğu için değişkeni kaldırmak mümkün değildir; en azından kullanıldığı yere odaklı bir geçici değişkendir. `getTotalBonus` da aynı mantıkla çalışır. Her filmden kazanılan bonus makbuza yazdırılmaz; makbuzun sonuna yalnız toplam bedel ve toplam bonus yazılır.

12.1 Performans tartışması

Soru: Burada bir performans sorunu yok mu? Eski halde `while`'daki tek geçişte hem toplam bedel hem bonus hesaplanıyordu.

Cevap: Eskiden liste bir kez dolaşılıyordu; şimdi üç kez dolaşılır: biri makbuzun her satırını basmak için (bu zaten vardı), ikisi de toplam bedel ve toplam bonus için. Hoca bunun kayda değer bir performans farkı yaratacağını düşünmediğini söyledi. Listeler milyonlarca kayıt içerse ve farklı sunuculardan, ağ üzerinden gelse bile, hele veriler sanallaştırılmış ayrı bir sunucuda duruyorsa performans sorunu en aza iner; ayrıca sistemin ana darboğazı büyük olasılıkla burası değildir.

Slayttaki bir notta `getTotalCharge` ve `getTotalBonus` içindeki `result` adlarının `totalCharge` ve `totalBonus` olmasının daha anlaşılır olacağı yazılıdır. Hoca bunun olabileceğini de olmayabileceğini de söyledi ve bu aşamada yapmadı; küçük bir iyileştirme olarak kaldı.

13. 7. ve 8. Adımlar: Hesapların Movie Sınıfına Taşınması

13.1 Sorun: başka nesnenin verisine göre karar vermek

Farklı kişiler koda baktığında düzenlemeye devam edilecek farklı noktalar görebilir. Fowler'ın ve ders notunun dikkat çektiği yer şudur: `getCharge` metodu `Rental`'a taşındı, ama bu metot hem `Movie`'ye erişiyor hem de içinde hâlâ bir `switch-case` bloğu var. `Rental` sınıfında olunmasına rağmen, ilişkili olunan bir nesnenin (filmin) fiyat kodu elde ediliyor. Bunu elde etmek tek başına sorun değildir, ama karar o fiyat kodu üzerinden veriliyor.

Hocanın ilkesi şudur: bir nesne kendi verileriyle ilgili kararları kendisi vermelidir; bir nesnenin verisiyle ilgili kararlar başka bir nesnede pek verilmemelidir. Sorumluluk, ilgili veriye en çok sahip olan yere taşınır; bölünebiliyorsa bölünür ve her kısım verisinin sahibi olan nesneye gider.

Filmin bedelini hesaplamakta iki parametre vardır: kaç gün kiralandığı ve filmin türü. Kira süresi bilgisi `Rental` sınıfındadır. Film türüne göre ödenmesi gereken miktarlar ise film türüne göre değiştiği için `Movie` sınıfında olmalıdır. O halde hesaplama `Movie`'ye taşınır. `Movie`'nin hesabı tamamlayabilmesi için filmin kaç gün kiralandığını bilmesi gerekir; bu bilgiyi `Rental` kendi sahip olduğu bilgi olarak parametreyle verir. Hoca konuşmayı şöyle canlandırdı: `Rental` filme "seni 3 gün kiraladılar" der ve 3 bilgisini verir; "buna göre ne kadar para ödenmesi gerektiğini hesapla" der; film de hesaplar.

13.2 7. adım: Movie.getCharge(int)

`fowler_06` ile `fowler_07` karşılaştırıldığında `Rental.getCharge()` "upuzun" halinden tek satıra iner (ekrandan, 00:57:30):

```
public double getCharge( ) {
    return _movie.getCharge( _daysRented );
}
```

`Movie` ise yeni bir metot kazanır (ekrandan, 00:58:05 ve 01:00:10):

```
public double getCharge( int daysRented ) {
    double result = 0;
    switch ( getPriceCode() ) {
    case Movie.REGULAR:
        result += 2;
        if (daysRented > 2)
            result += (daysRented - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
        result += daysRented * 3;
        break;
    case Movie.CHILDRENS:
        result += 1.5;
        if (daysRented > 3)
            result += (daysRented - 3) * 1.5;
        break;
    }
    return result;
}
```

`switch-case` hâlâ durur, ama artık sınıfın kendi bilgisi üzerinde karar verir; bu yüzden sakıncası azalmıştır. Geçici değişken dahil başka bir değişiklik yoktur. Tek fark, `Rental`'dayken ilgili üyelere getter metotlarıyla erişilirken burada kira süresinin parametre olarak gelmesi ve `Movie`'nin kendi fiyat kodunu kendi metoduyla almasıdır. Testler tekrarlandı ve sorun çıkmadı.

13.3 8. adım: Movie.getBonus(int)

Bedel için yapılan şey bonus için de yapıldı. `Movie` içinde kaç gün kiralandığı bilgisini alan bir `getBonus` metodu oluşturuldu (ekrandan, 01:00:10):

```
public int getBonus( int daysRented ) {
    if ( getPriceCode() == Movie.NEW_RELEASE && daysRented > 1 )
        return 2;
    else
        return 1;
}
```

`Rental.getBonus()` artık kira süresini filme gönderip sonucu kendisini çağıran `Customer`'a iletir: `return _movie.getBonus( _daysRented );` (ekrandan, 01:05:58). Test durumlarında yine bir değişiklik gerekmedi.

14. 9. Adım: Etkileşim Şeması ve getMovieTitle

Hoca, ilk adımdan beri gösterdiği etkileşim şemasının anlam kazandığı yere gelindiğini söyledi. Slayt 61'deki 8. adımın sınıf şeması ve etkileşim şeması şöyledir (ekrandan, 01:02:50):

```
Movie    - _priceCode : int
         + getCharge( daysRented : int ) : double
         + getBonus( daysRented : int ) : int
   ^ 1
   | 1
Rental   - _daysRented : int
         + getCharge( ) : double
         + getBonus( ) : int
   ^ *
   | 1
Customer + statement( )
         + getTotalCharge( ) : double
         + getTotalBonus( ) : int
```

```
Kullanıcı -> aCustomer : statement()
  aCustomer *[for all rentals]:
    aCustomer -> aRental : getMovie()
    aCustomer -> aMovie  : getTitle()          <- doğrudan Customer -> Movie çağrısı
    aCustomer -> aRental : getCharge()
      aRental -> aMovie  : getCharge(_daysRented)
    (Tek kira için ücret hesabı ve makbuz satırı basımı tamamlanır)
  aCustomer -> aCustomer : getTotalCharge()
    *[for all rentals] aCustomer -> aRental : getCharge()
      aRental -> aMovie : getCharge(_daysRented)
  aCustomer -> aCustomer : getTotalBonus()
    *[for all rentals] aCustomer -> aRental : getBonus()
      aRental -> aMovie : getBonus(_daysRented)
  (Makbuz basımı tamamlanır)
```

Şemada `Customer` hep `Rental` ile, `Rental` da hep `Movie` ile konuşur. Tek istisna, hocanın ekranda yuvarlak içine aldığı `getTitle()` çağrısıdır: `Customer` ile `Movie` arasında giden doğrudan tek bağlantı budur. Hoca bu "çıkıntının" dikkatini çektiğini söyledi. Bu çıkıntı kaldırılırsa `Customer`'ın `Movie` ile doğrudan hiçbir ilişkisi kalmaz, yalnız dolaylı ilişkisi kalır.

`Customer`'ın `Movie` ile tek işi filmin adını almaktır; bütün hesaplamalar artık `Rental` ve `Movie` sınıflarına dağıtılmıştır. `Customer` `Rental`'ın `getMovie()` metodunu çağırıyor, ama dönen nesneyle başlığı almaktan başka iş yapmıyor. Makbuza hangi filmin kiralandığını yazmak zorunda olduğu için başlığa ihtiyacı vardır. Bu yüzden `Rental`'a bir metot eklenir (ekrandan, 01:05:58):

```
public String getMovieTitle( ) {
    return _movie.getTitle( );
}
```

`Customer.statement` içinde `aRental.getMovie().getTitle()` yerine artık `aRental.getMovieTitle()` çağrılır (ekrandan, 01:10:00). Başka değişiklik yoktur. Görünüşte küçük bir değişikliktir, ama sonuçta katmanlı bir ilişki kurulur: `Customer` yalnız `Rental` ile, `Rental` da `Movie` ile konuşur. Bu da bağlaşımı azaltır. Hoca, daha önce sınıf şemasında görünmese de `Customer` ile `Movie` arasında dolaylı bir ilişki olduğunu, artık kaynak kodda da bu ilişkinin kalmadığını söyledi.

> **Ek bilgi (derste anlatılmadı):** Bir nesnenin yalnız doğrudan ilişkili olduğu nesnelerle konuşması, nesnelerin içinden geçerek uzak nesnelere erişmemesi ilkesi literatürde büyük olasılıkla "Law of Demeter" (en az bilgi ilkesi) adıyla bilinir. `aRental.getMovie().getTitle()` gibi zincirleme çağrılar bu ilkenin ihlaline örnektir; Fowler bu duruma "Message Chains" kod kusuru, düzeltmesine "Hide Delegate" adını verir.

Hoca ayrıca ders notlarında yer almayan, sonradan eklediği bir öneriyi sözlü olarak aktardı: `statement`'ın sonundaki iki satırlık makbuz altlığı (footer) kısmı da bir alt iştir ve `printFooter` gibi bir metoda çıkarılabilir (Extract Method). Son hal `fowler_09` paketidir.

15. 10. Adım: Çok Biçimlilik ya da Price Sınıfları

15.1 İdeal tasarım: soyut Movie ve alt sınıfları

Hoca, işin başında kendisi bir tasarım yapsaydı şunu yapacağını söyledi: `getCharge` ve `getBonus` metotları film nesnelerinde olurdu ve soyut üst sınıfta soyut metot olarak imzaları bulunurdu. Slayt 64 bu tasarımı verir (ekrandan, 01:07:50):

```
Onuncu değişiklik: getCharge metodundaki switch/case ve getBonus metodundaki
if deyimleri yerine çokbiçimlilik ilişkisi kullanılması (kalıtım ve soyut metotlar).

Customer (+ statement(), + getTotalCharge() : double, + getTotalBonus() : int)
   1 ---- * Rental (- _daysRented : int; + getCharge() : double,
                    + getBonus() : int, + getMovieTitle() : String)
Rental 1 ---- 1 <<abstract>> Movie
          + getCharge( daysRented : int ) : double {abstract}
          + getBonus( daysRented : int ) : int {abstract}
Movie <|-- MovieNewRelease, MovieRegular, MovieChildrens
```

Soru: `Movie` neden bir arayüz (`interface`) değil de soyut sınıftır?

Cevap: `Movie`'nin bir adı (title) üye alanı vardır; ad için getter metodu, kurucusu gibi gövdesi doğrudan yazılabilen metotları vardır. Bu yüzden bu iş için en uygunu soyut sınıftır (abstract class).

Ama tasarım bu hale getirilirse geriye doğru uyumluluk kaybedilir. Dersin başındaki tartışmaya dönülür: geriye doğru uyumluluğu korumak çok önemliyse saf (pure) refactoring yapılmış olur ve `fowler_09`'da durulur. Uyumluluk o kadar önemli değilse tasarımı bu hale getirmek çok daha doğrudur.

15.2 Alternatif: kalıtım yerine sahiplik (Price)

Hoca, yeni film alt sınıfları oluşturmadan başka bir şey yapılıp yapılamayacağını sordu ve yapılabileceğini söyledi. `switch-case`'i ortaya çıkaran en büyük sebep farklı film türleridir. Kalıtım yerine sahiplik ilişkisi kullanılabilir. Olay filmin fiyatıyla ilgili olduğu için fiyatla ilgili işler ayrı bir `Price` sınıfına verilir; `Movie`'ye bir `price` üyesi eklenir.

Hoca bunun tam olarak Strategy kalıbı olmadığını, ama Strategy'nin "bir işi yapmanın birden fazla yolu varsa en iyisine karar ver" mantığına benzediğini söyledi; filmlerin farklı hesaplama çeşitleri olacağı düşünülürse Strategy akla gelebilir. Asıl önemli olan, bir kalıp akla gelmeden "bunu nasıl daha iyi modellerim, kalıtım yerine sahiplik türünden bir şey yapılamaz mı" diye düşünmektir. Hoca bunu farklı filmlere farklı fiyat etiketleri yapıştırmaya benzetti: `Movie`'ye hangi `Price` alt sınıfı verilirse, "artık sen o türden fiyatlanıyorsun" denmiş olur.

> **Ek bilgi (derste anlatılmadı):** Fowler, Refactoring kitabının 1. bölümünde bu adımı büyük olasılıkla "Replace Type Code with State/Strategy" olarak adlandırır ve fiyatın bir filmin ömrü boyunca değişebilen bir durum olduğunu düşünerek yapıyı State kalıbına yakın görür. Ders 11'de bu düzenlemenin, kalıtım mümkün olmadığında ya da nesnenin türü çalışma sırasında değişebildiğinde tercih edildiği görülmüştü: bir film "yeni çıkan" kategorisinden "normal" kategorisine geçtiğinde nesne yeniden oluşturulmadan yalnız `_price` üyesine yeni bir nesne atanır.

Bu yapıda test durumlarının arayüzü değişmez; işler yine `getBonus` ve `getCharge` metotları üzerinden yapılır. Ama `Movie`'deki bu metotlar işlerini esas olarak `price` üye alanına yaptırır. `fowler_09` ile `fowler_10_alt` karşılaştırıldığında `Customer`'da değişiklik yoktur; değişiklik `Movie`'dedir (ekrandan, 01:20:00):

```
public class Movie {
    private String _title;
    private Price _price;
    public static final int CHILDRENS = 2;
    public static final int NEW_RELEASE = 1;
    public static final int REGULAR = 0;
    public Movie(String title, int priceCode) {
```

```
        _title = title;
        setPriceCode( priceCode );
    }
    public int getPriceCode() {
        return _price.getPriceCode();
    }
    ...
}
```

Geriye doğru uyumluluğu korumak için sabitler ister istemez kalır; bu sabitler test kodlarında bile kullanılmaktadır. `Movie`'ye bir `Price` verilecektir, ama hangi `Price`'ın oluşturulacağı yine bu tip kodlarına göre belirlenir: `CHILDRENS` verilmişse `PriceChildrens`, `NEW_RELEASE` verilmişse `PriceNewRelease`, `default` durumunda `PriceRegular` nesnesi `_price` üyesine atanır. Yani hâlâ bir `switch-case` vardır, ama çok daha kısadır: eskiden içinde bir sürü hesaplama vardı, şimdi yalnız bir ilklendirme kodu içerir.

`Price` soyut sınıfı şöyledir (ekrandan, 01:16:15):

```
public abstract class Price {
    public abstract double getCharge( int daysRented );
    public abstract int getPriceCode( );
    public int getBonus( int daysRented ) {
        return 1;
    }
}
```

`getCharge` ve `getPriceCode` soyuttur. `getBonus` ise burada gövdesiyle yazıldı: varsayılan davranış her kiralamadan 1 puan kazandırmaktır ve bu gövde birçok alt sınıfa doğrudan kalıtımla aktarılabilir. Yalnız yeni çıkan filmler bu metodu ezer (override). Diğer metotlarda böyle ortak bir gövde olmadığı için onlar soyut kalır. Her alt sınıfın `getCharge`'ı, eski `switch`'teki ilgili `case`'in kodunun taşınmasıyla oluşur; `getPriceCode` da ilgili sabiti döndürür, çünkü eski kodlar fiyat kodunu sorgulamaya devam edebilmelidir. `PriceNewRelease` örneği (ekrandan, 01:17:35):

```
public class PriceNewRelease extends Price {
    public double getCharge( int daysRented ) {
        double result = 0;
        result += daysRented * 3;
        return result;
    }
    public int getBonus( int daysRented ) {
        if( daysRented > 1 )
            return 2;
        else
            return 1;
    }
    public int getPriceCode() {
        return Movie.NEW_RELEASE;
    }
}
```

> **Ders sonrası not:** Hoca `getBonus`'un ezilmesini anlatırken (01:16:30) "kira günü 2'den büyükse 2 puan" dedi, sonra "kaçtan büyüktüyse" diye düzeltmeye yöneldi. Ekrandaki kodda koşul `daysRented > 1`'dir (ekrandan, 01:17:35); yani yeni çıkan film 1 günden uzun kiralanırsa 2 puan kazanılır. Bu, başlangıç kodundaki kuralla da aynıdır.

Sonuçta `Rental` içinde hesaplama yapılırken hesaplamalar `switch` içinde yer almaz; `switch` yalnız başlangıçta bir ilklendirme kodu içerir. `Rental`'da ve `Customer`'da kayda değer bir değişiklik yoktur; yapılan refactoring eylemleri `Movie` ile ilgilidir. Hoca `fowler_11_alt` paketindeki değişikliği anlatmadı, incelemeyi öğrencilere bıraktı.

> **Ek bilgi (derste anlatılmadı):** Ajan, başlangıç kodundaki bedel ve bonus kuralları ile son haldeki (`Movie.getCharge(int)`, `getBonus(int)` ve `PriceNewRelease`) kuralların 0-29 gün ve üç fiyat kodu için aynı sonucu verdiğini python ile karşılaştırarak

denetledi. Bu, adımların davranışı korumak için tasarlandığını sayısal olarak da doğrular; derste ise bu denetim JUnit testleriyle yapıldı.

16. Karşıt Kalıplar (Anti-Patterns)

Ders notlarındaki son slayt (slayt 65) karşıt kalıplar konusudur. Hoca "anti-pattern"ı "karşıt kalıp" diye çeviriyor. Tasarım kalıpları iyi tasarım yapma konusunda ipuçları verir; karşıt kalıplar ise kötü tasarımın nasıl yapıldığını gösterir, böylece yanlışın ne olduğu görülüp yapılmaz.

Hoca konuyu derste görülenlerle ilişkilendirdi. Kod kusurları kod düzeyindeki küçük kusurlardı. Derste ağırlıklı olarak tasarım kalıpları görüldü; bunlar daha büyük ölçekte ve doğru tasarım yapmayı sağlıyordu. Bu sırada "kafamıza göre" yapılan kusurlu tasarım örnekleri de görüldü; işte bu tür hatalara verilen adlar karşıt kalıplardır. Karşıt kalıplar, kod kusurlarının daha üst ölçekteki benzerleri gibi düşünülebilir. Daha güzel bir tanımla: karşıt kalıp, bir tasarım kalıbı gibi görünen ama sizi kötü bir modellemeye yönlendiren çözümdür.

Slayttaki içerik şöyledir (ekrandan, 01:22:30):

- Antipattern nedir?
  - Karşıt kalıp çözüm bir tasarım kalıbı gibi görünür ancak öyle değildir.
  - Karşıt kalıplar aslında tasarım kalıplarının karşıtları olarak yazılım geliştirme sürecinde tekrar eden bazı yanlışları ifade ederler.
  - Karşıt kalıpları bilmek, yazılım geliştirme sürecinde karşılaşılabilecek ciddi problemleri önceden tahmin edebilmeyi ve tedbir almayı kolaylaştırır.
- Karşıt kalıp ile kod kusuru farkı:
  - Karşıt kalıplar tasarım düzeyine, kod kusurları gerçekleme düzeyine daha yakındır.
  - Karşıt kalıplar yazılım yaşam döngüsünün diğer evreleri ile de ilişkilidir.
- Karşıt kalıp türleri: karşıt kalıplar üç ayrı grupta incelenir (slaytta "henüz müfredata eklenmedi" notu vardır):
  - Yazılım geliştirme karşıt kalıpları
  - Yazılım mimarisi karşıt kalıpları
  - Yazılım proje yönetimi karşıt kalıpları

Hoca, karşıt kalıpların yalnız koda yaklaşmadığını, yazılım geliştirme yaşam döngüsünde ve yazılım proje yönetiminde karşılaşılan hataları da kategorize ettiğini söyledi. Konunun ayrı bir kitabı da vardır.

Hoca dersi beğenenlere ve konuyu biraz daha çalışmak isteyenlere şu yönlendirmeleri yaptı: derste bütün kod kusurları görülmedi; kalanlar Fowler'ın kitabından okunabilir. Literatürde Fowler'ın önerdiklerinden başka kod kusurları ve refactoring eylemleri de vardır. Karşıt kalıplar konusuna da bakılabilir; kitabı almak şart değildir, konuyu kategorilere ayırarak anlatan bir web sitesi vardır. Ders sonunda hoca bu siteyi (sourcemaking.com) açtı (ekrandan, 01:40:00). Sitede tasarım kalıpları yaratımsal (creational), yapısal (structural) ve davranışsal (behavioral) diye anlatılır; karşıt kalıplar arasında "Spaghetti Code" gibi ilginç adlar vardır; refactoring ve kod kusurları da bulunur. Sitede kod kusurları kategorilere ayrılmıştır; örneğin "Bloaters" grubunda Long Method, Large Class, Primitive Obsession, Long Parameter List ve Data Clumps yer alır. Hoca Fowler'ın ve kendisinin kod kusurlarını bu şekilde kategorilere ayırarak anlatmadığını belirtti; Data Clumps gibi derste görülmeyenlere bakılabileceğini, Temporary Field'ın da aslında geçici değişkenle ilgili olduğunu söyledi.

Örnek Sorular

Bu derste ekranda ayrı bir soru ya da alıştırma gösterilmedi. Aşağıdaki sorular derste sözlü olarak sorulan ya da hocanın bu dersteki örnek üzerinden tarif ettiği sorulardır.

Soru 1 (derste sözlü, 01:29:30 - 01:36:41): Bir A4 sayfanın önünü ve arkasını dolduracak uzunlukta bir kod verilir (örneğin video kiralama örneğindeki `Customer`, `Movie` ve `Rental` sınıflarının ilk halleri alt alta). (a) Koddaki kod kusurlarını işaretleyip literatürdeki özgün adlarıyla yazınız; her biri için uygulanacak refactoring eyleminin adını yine literatürdeki haliyle yazınız. (b) Bütün düzenlemeler yapıldığında varılacak tasarımın UML sınıf şemasını çiziniz.

> **Çözüm (cevap biçimi derste verildi; liste bu dersteki adımlardan kuruldu, "Feature Envy" ve "Switch Statements" adlarını bu adımlarla ajan eşleş-**

tirdi): Hoca cevabın biçimini şöyle gösterdi: kusurlu yerler kod üzerinde yuvarlak içine alınıp numaralanır, cevap kâğıdında liste halinde yazılır; örneğin "Numara 1: Long Method; çözüm için Extract Method", "Numara 3: Duplicate Code (üç ayrı yerde); çözüm için Extract Method". Kod yazmaya gerek yoktur; kusur olmayan yeri kusur diye işaretlememek gerekir. Video kiralama kodu için bu dersteki adımlardan çıkan liste:

- `Customer.statement()` çok uzun: Long Method → Extract Method (`amountFor`, sonra `getTotalCharge`, `getTotalBonus`).
- `amountFor` müşterinin verisini kullanmıyor, `Rental` ve `Movie` verisini kullanıyor: Feature Envy → Move Method (`Rental.getCharge`, sonra `Movie.getCharge(int)`); bonus hesabı için de aynısı.
- `thisAmount` bir kez atanıp değişmiyor: Replace Temp with Query.
- `each` gibi anlamsız adlar: yeniden adlandırma (Rename).
- `aRental.getMovie().getTitle()`: `Customer`'dan `Movie`'ye doğrudan çağrı → `Rental.getMovieTitle()`.
- Fiyat koduna göre `switch-case`: Switch Statements → Replace Conditional with Polymorphism / Replace Type Code with Subclasses ya da State/Strategy (`Price`).

Varılacak tasarım: `Customer` (statement, getTotalCharge, getTotalBonus) 1 → * `Rental` (_daysRented; getCharge, getBonus, getMovieTitle) * → 1 `Movie`; `Movie` ya soyut sınıftır ve `MovieRegular`, `MovieNewRelease`, `MovieChildrens` alt sınıflarına sahiptir, ya da 1 → 1 bir `Price` üyesi tutar ve `Price`'ın `PriceRegular`, `PriceNewRelease`, `PriceChildrens` alt sınıfları vardır.

Soru 2 (derste sözlü, 00:38:47): `Rental` sınıfı yeni bir `getCharge` metodu kazandı. Bu metot için ayrı bir test durumu oluşturmak gerekir mi?

Çözüm (derste verildi): Beyaz kutu yaklaşımında, yani birimin iç yapısına göre sınama yapılıyorsa evet. Kara kutu yaklaşımında hayır: yalnız verilen girdilere karşı beklenen çıktılara bakılır ve arayüz değişmediği için mevcut testler yeterlidir. Derste daha az masraflı olan kara kutu yaklaşımıyla devam edildi.

Soru 3 (derste sözlü, 00:39:52 - 00:41:06): `statement` metodundaki `totalAmount`, `frequentRenterPoints` ve `thisAmount` geçici değişkenlerinden hangisi Replace Temp with Query ile kaldırılabilir? Neden?

Çözüm (derste verildi): Yalnız `thisAmount`: bir kez değer alır ve geçerlilik süresince değişmez; kullanıldığı iki yerde `aRental.getCharge()` çağrılabilir. `totalAmount` ve `frequentRenterPoints` kümülatif toplamlardır, bir yerde biriktirilmeleri gerekir; ancak sonradan `getTotalCharge()` ve `getTotalBonus()` metotlarının içine alınarak `statement`'tan çıkarıldılar.

Soru 4 (derste sözlü, 00:50:00 - 00:51:30): `getTotalCharge` ve `getTotalBonus` eklendikten sonra kira listesi bir yerine üç kez dolaşılıyor. Bu bir performans sorunu mudur?

Çözüm (derste verildi): Hocaya göre kayda değer bir fark yaratmaz. Liste çok büyük olsa ya da ağ üzerinden gelse bile sistemin asıl darboğazı büyük olasılıkla burası değildir; kazanılan okunabilirlik ve sorumluluk dağılımı daha değerlidir.

Soru 5 (derste sözlü, 01:08:08): İdeal tasarımda `Movie` neden `interface` değil de soyut sınıf olarak tanımlanır?

Çözüm (derste verildi): `Movie`'nin bir ad (title) üye alanı ve gövdesi yazılabilen metotları (kurucu, `getTitle`) vardır. Arayüz bunları taşıyamaz; `getCharge` ve `getBonus` soyut, diğerleri gövdeli olacak biçimde soyut sınıf en uygunudur.

Dersin Toparlaması

Bu derste kod kusurları ve refactoring konusu, Fowler'ın video kiralama örneği üzerinde bütüncül olarak uygulandı. Başlangıçta `Customer.statement()` bütün işi yapan uzun bir metottu. Önce `switch` bloğu `amountFor` metoduna çıkarıldı, adlar düzeltildi, metot verisinin sahibi olan

`Rental`'a `getCharge` adıyla taşındı ve gereksiz `thisAmount` değişkeni sorgu çağrısıyla değiştirildi. Aynı işlemler bonus hesabı için yapıldı; toplamlar `getTotalCharge` ve `getTotalBonus` metotlarına alındı. Ardından film türüne bağlı hesaplar `Movie`'ye taşındı, kira süresi parametre olarak geçirildi. Etkileşim şeması `Customer`'dan `Movie`'ye giden tek doğrudan çağrıyı gösterdi; `getMovieTitle` ile bu çağrı kaldırılıp katmanlı bir yapı kuruldu. Her adımda arayüz değişmediği için aynı JUnit testleri hiç değiştirilmeden geçti.

Son olarak `switch`'in kendisi tartışıldı. Soyut `Movie` ve film türü alt sınıflarıyla çok biçimli tasarım en temiz çözümdür, ama `Movie` kurucusundaki fiyat kodu ve public sabitler yüzünden geriye doğru uyumluluğu bozar. Kalıtım yerine sahiplikle kurulan `Price` hiyerarşisi uyumluluğu korur ve `switch`'i yalnız bir ilklendirme koduna indirger. Ders, tasarım düzeyindeki yanlışları adlandıran karşıt kalıplar (anti-patterns) konusuna ve konuyu ileri götürmek isteyenler için kaynak önerilerine kısa bir girişle kapandı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.